

LAYER 3

THE INTELLIGENCE & OPTIMIZATION LAYER

Integration Gates Specification

for Agentic Coding

DAO Governance Architecture
Self-Evolving & Self-Optimizing Quadratic Voting Module

Derived from the DAO Governance Architecture Document
First-Principles Design | OODA Loop Integration | Safety-First Constraints

DOCUMENT OVERVIEW

This specification defines the **8 Integration Gates** for Layer 3 (The Intelligence & Optimization Layer) of a Self-Evolving DAO Governance Architecture based on Quadratic Voting. Each gate represents a critical checkpoint in the autonomous adaptation cycle, enforcing safety, validation, and constraint boundaries before any parameter adjustment can propagate through the system.

OODA Phase	Gate ID	Gate Name	Layer Interface
ACT	—	Users submit proposals & vote	Layer 2
OBSERVE	GATE 1	Data Ingestion Gate	Layer 4 → Layer 3
ORIENT	GATE 2	KPI Analysis Gate	Internal
ORIENT	GATE 3	Optimization Engine Gate	Internal (ML)
ADAPT	GATE 4	Safety Bounds Gate	Internal
ADAPT	GATE 5	Treasury Isolation Gate	Internal (Firewall)
ADAPT	GATE 6	Timelock Broadcast Gate	Layer 3 → Layer 5
VERIFY	—	Timelock delay + Veto mechanism	Layer 5
LEARN	GATE 7	Feedback Loop Gate	Layer 2 → Layer 3
GOVERN	GATE 8	Model Versioning & Governance Gate	Meta-Governance

CORE ARCHITECTURAL CONSTRAINTS (NON-NEGOTIABLE)

Constraint 1: Layer 3 cannot directly touch treasury funds or force-execute proposals. It is a parameter-recommendation engine only.

Constraint 2: Layer 3 can only adjust predefined variables within strict, hardcoded upper and lower bounds. No parameter can escape its mathematical cage.

Constraint 3 (The Safety Principle): Any automated parameter adjustment must be mathematically bounded to prevent runaway feedback loops or malicious capture.

GATE 1: DATA INGESTION GATE

Interface: Layer 4 (Data & Oracle Layer) → Layer 3 (Intelligence Layer)

Purpose: To serve as the single, validated, cryptographically verified data entry point into the Intelligence Layer. This gate ensures that all downstream optimization decisions are based on tamper-proof, multi-source verified data. Without this gate, the entire self-evolution mechanism is vulnerable to data-manipulation attacks against the optimization engine.

Inputs

- On-chain metrics: voter turnout, token price, treasury velocity, quadratic cost distributions
- Quantifiable off-chain data: sentiment scores, market indicators, social signals
- Historical voting distributions: credit allocation patterns, preference intensity curves
- Proposal outcomes: pass/fail status, execution results, treasury impact

Validation Rules

- ✓ Multi-source aggregation check — minimum 3 independent oracle sources required
- ✓ Cryptographic signature verification — each oracle signature must be valid and from a trusted source
- ✓ Timestamp freshness — on-chain data < 1 hour old, off-chain data < 24 hours old
- ✓ Anomaly detection — Z-score > 3.0 triggers manual review flag (data accepted but flagged)

Agentic Coding Interface

```
function: ingest_governance_data(data_payload, oracle_signatures)

returns: ValidatedDataFrame | RejectionReason

preconditions:
- oracle_signatures.length >= 3
- all_signatures_valid == true
```

Failure Modes

- **Insufficient Sources:** Rejected immediately if fewer than 3 oracle signatures provided.
- **Untrusted Oracle:** Rejected if any signature originates from a non-whitelisted oracle.
- **Stale Data:** Rejected if data exceeds freshness thresholds.
- **Anomalies:** Accepted with escalation flag for manual review; does not block the pipeline but logs for audit.

GATE 2: KPI ANALYSIS GATE

Interface: Internal Processing (Layer 3)

Purpose: To transform the raw, validated governance data into a standardized set of Key Performance Indicators (KPIs) that the optimization engine can reason about. This gate enforces deterministic computation — the same input must always produce the same KPI output — ensuring reproducibility and auditability of all optimization decisions.

KPI Definitions

- $\text{voter_apathy_rate} = 1 - (\text{actual_voters} / \text{eligible_voters})$
— Measures participation decay. High apathy signals need for quorum or incentive adjustment.
- $\text{proposal_success_rate} = \text{passed_proposals} / \text{total_proposals}$
— Measures governance throughput. Too low suggests barriers are too high; too high suggests insufficient scrutiny.
- $\text{quadratic_efficiency} = \text{actual_cost} / \text{theoretical_optimal_cost}$
— Measures how close voting patterns are to the ideal quadratic distribution. Deviations indicate Sybil risk or strategic voting.
- $\text{treasury_impact_velocity} = \text{treasury_outflow} / \text{time_window}$
— Measures speed of treasury deployment. Used to detect spending spikes that may require slower voting periods.
- $\text{credit_utilization_rate} = \text{credits_spent} / \text{credits_allocated}$
— Measures engagement intensity. Low utilization suggests voters are hoarding credits or lack compelling proposals.

Validation Rules

- ✓ All KPIs must be computed exclusively from GATE_1 validated data
- ✓ KPI computation must be deterministic — same input always yields identical output
- ✓ KPI values must be bounded $[0, 1]$ or within historical ± 5 standard deviations
- ✓ Minimum time window of 7 days required for statistical significance

Agentic Coding Interface

```
function: compute_kpis(validated_dataframe, time_window)
returns: KPIMetrics | ErrorCode
preconditions:
- validated_dataframe.not_empty
- time_window >= 7_days
```

GATE 3: OPTIMIZATION ENGINE GATE

Interface: Internal ML Model Execution (Layer 3)

Purpose: To generate parameter adjustment recommendations using applied AI/ML. This is the 'brain' of the self-evolution mechanism — it analyzes historical voting data, identifies inefficiencies, and proposes tuned parameter values to maximize governance efficiency and voter satisfaction. However, it operates under strict constraints: it can only recommend, never execute, and only within predefined bounds.

Tunable Parameters (with Hardcoded Bounds)

Parameter	Lower Bound	Upper Bound	Default	Max Δ/Epoch
quorum_threshold	0.05	0.95	0.51	0.10
voting_duration_blocks	1,000	50,000	10,000	5,000
voice_credit_regeneration_rate	1	100	10	5
proposal_deposit_amount	100	10,000	1,000	500
dynamic_quorum_multiplier	0.5	2.0	1.0	0.2

Optimization Logic Examples

- High voter_apathy (> 0.5) → Lower quorum_threshold, increase voice_credit_regeneration_rate
- Low proposal_success_rate (< 0.2) → Reduce proposal_deposit_amount, lower dynamic_quorum_multiplier
- High treasury_impact_velocity → Extend voting_duration_blocks for larger proposals
- Low quadratic_efficiency → Adjust dynamic_quorum_multiplier to better reflect true preference intensity

Validation Rules

- ✓ All recommended values must fall within hardcoded bounds
- ✓ Recommendation must include confidence score $\in [0, 1]$
- ✓ Confidence < 0.7 triggers ConservativeFallback (no changes applied)
- ✓ Model drift detection: if input distribution shift exceeds threshold, flag for retraining
- ✓ Only registered and governance-approved model versions may be active

Agentic Coding Interface

```
function: optimize_parameters(kpi_metrics, model_version)
returns: ParameterAdjustmentProposal | ConservativeFallback
preconditions:
- kpi_metrics.validated (from GATE_2)
- model_version.registered and active (from GATE_8)
```

GATE 4: SAFETY BOUNDS GATE

Interface: Internal — The Constraint Enforcer

Purpose: To enforce absolute mathematical safety boundaries on all parameter adjustments. This is the most critical gate in the safety chain. Even if the optimization engine produces a recommendation, this gate has the final authority to reject it if any safety constraint is violated. It prevents runaway feedback loops, malicious parameter capture, and cascading failures.

Safety Checks

- ✓ Parameter value $\in$ [hardcoded_lower, hardcoded_upper] — absolute bounds check
- ✓ Rate of change $\leq$ max_delta_per_epoch — prevents violent oscillations
- ✓ Cumulative change over N epochs $\leq$ max_cumulative_delta — prevents death-by-a-thousand-cuts
- ✓ No adjustment can reduce quorum below minimum_safety_quorum (e.g., 0.10) — prevents governance capture
- ✓ No adjustment can increase voting_duration beyond max_reasonable_duration — prevents proposal gridlock
- ✓ Treasury impact of any parameter change must be ≤ 0 — Layer 3 cannot create positive treasury outflows

Validation Rules

- ✓ ALL safety checks must pass before any parameter can be adjusted — no partial approvals
- ✓ Failed safety check triggers automatic rejection with detailed, immutable violation log
- ✓ Safety violations are logged to an append-only audit trail for post-mortem analysis
- ✓ If a safety violation is detected, the system emits an alert to the Layer 5 veto multisig

Agentic Coding Interface

```
function: enforce_safety_bounds(parameter_proposal, current_params,
                                safety_config)
```

```
returns: SafeParameterSet | ViolationReport
```

```
preconditions:
```

- parameter_proposal.exists (from GATE_3)
- safety_config.loaded and immutable

Example Violation Scenarios

- Proposed quorum_threshold = 0.03 → REJECTED (below safety floor of 0.05)
- Proposed voting_duration = 60,000 blocks → REJECTED (above upper bound of 50,000)
- Quorum delta of 0.15 in one epoch → REJECTED (exceeds max_delta_per_epoch of 0.10)
- Cumulative quorum reduction of 0.30 over 3 epochs → REJECTED (exceeds cumulative limit)

GATE 5: TREASURY ISOLATION GATE

Interface: Internal — The Firewall

Purpose: To provide an absolute, cryptographic guarantee that Layer 3 cannot access, move, or influence treasury funds in any way. This gate operates as a formal firewall between the intelligence layer and the treasury. It is the implementation of Core Constraint 1 at the transaction level. Even if all other gates were compromised, this gate ensures the treasury remains untouchable by the optimization engine.

Enforcement Mechanisms

- ✓ Parameter adjustments are data-only — no transfer, approve, or call-value operations permitted
- ✓ All generated transactions must use function selectors from an explicit whitelist
- ✓ Any transaction with 'to' address matching a treasury contract is automatically rejected
- ✓ Static bytecode analysis scans for embedded treasury addresses before any transaction is signed
- ✓ Transaction value field must be exactly 0 — no ETH/token transfers possible

Allowed Function Selectors (Whitelist)

- 0x12345678 — setQuorumThreshold(uint256)
 - 0x87654321 — setVotingDuration(uint256)
 - 0xabcdef01 — setCreditRegenerationRate(uint256)
 - 0x10fedcba — setProposalDeposit(uint256)
 - 0x55667788 — setDynamicQuorumMultiplier(uint256)
- [Any selector not in this set is automatically rejected]

Validation Rules

- ✓ Bytecode analysis must confirm zero CALL instructions targeting treasury contracts
- ✓ Transaction value must be exactly 0 wei
- ✓ Function selector must exist in allowed_parameter_functions whitelist
- ✓ Any deviation triggers immediate rejection and security alert escalation

Agentic Coding Interface

```
function: verify_treasury_isolation(parameter_tx, treasury_addresses)
```

```
returns: IsolatedTx | Rejection
```

```
preconditions:
```

```
- treasury_addresses.defined and frozen at deployment
```

```
- parameter_tx.generated by GATE_4 approved SafeParameterSet
```

GATE 6: TIMELOCK BROADCAST GATE

Interface: Layer 3 → Layer 5 (Failsafe / Veto Layer)

Purpose: To submit validated, safety-checked, treasury-isolated parameter adjustments to the Layer 5 timelock. This is the final outbound gate of the adaptation phase. Once a parameter set passes through this gate, it enters a mandatory delay period during which the Layer 5 veto mechanism can review and potentially cancel the adjustment. This gate ensures no parameter change takes effect immediately — human oversight always has a window to intervene.

Broadcast Process

1. Package safe parameters into Layer 5 compatible format (ABI-encoded call data)
2. Attach comprehensive justification: KPI analysis summary, confidence score, expected impact, model version
3. Submit to timelock smart contract with mandatory delay_period
4. Emit on-chain event for veto multisig notification
5. Enter pending state until delay expires or veto is triggered

Validation Rules

- ✓ Justification must be non-empty and human-readable (minimum 100 characters)
- ✓ Delay period must be $\geq$ minimum_timelock_duration (e.g., 10,080 blocks $\approx$ 7 days)
- ✓ Duplicate adjustment detection via parameter hash comparison — prevents spam
- ✓ Emergency flag: if critical safety threshold is breached, can trigger expedited review pathway
- ✓ All adjustments must reference the GATE_4 SafeParameterSet hash for audit trail

Agentic Coding Interface

```
function: broadcast_to_timelock(safe_params, justification, delay_blocks)

returns: TimelockTxHash | BroadcastError

preconditions:

- safe_params.validated (from GATE_4)
- justification.length >= 100_chars
- delay_blocks >= MIN_TIMELOCK_BLOCKS
```

Timelock Payload Structure

```
{
  "parameter_adjustments": [
    {
      "param_name": "quorum_threshold",
      "proposed_value": "0.40",
      "current_value": "0.51",
      "confidence_score": 0.85
    }
  ],
  "justification": "High voter apathy (72%) detected...",
  "delay_blocks": 10080,
  "effective_block": "current_block + 10080",
  "kpi_snapshot": { ... }
```


GATE 7: FEEDBACK LOOP GATE

Interface: Layer 2 (State & Execution) → Layer 3 (Intelligence)

Purpose: To close the learning loop by capturing the real-world outcomes of parameter adjustments. After a parameter set has been active for a full governance cycle, this gate measures its effectiveness and feeds that data back into the optimization model. This is what makes the system genuinely self-evolving rather than merely self-adjusting — it learns from its own decisions.

Feedback Data Captured

- Actual proposal outcomes vs. model-predicted outcomes — prediction accuracy metric
- Voter turnout delta after parameter adjustments — measures engagement response
- Credit utilization patterns post-adjustment — reveals strategic behavior shifts
- Time-to-consensus metrics — efficiency of the new parameter regime
- Quadratic cost distribution changes — detects Sybil or collusion patterns

Validation Rules

- ✓ Feedback must be sourced exclusively from confirmed (finalized) proposals — no pending data
- ✓ Data must be timestamped and attributed to a specific parameter set version
- ✓ Outlier detection: extreme deviations ($> 3\sigma$ from expected) trigger model retraining consideration
- ✓ Feedback data must pass through GATE_1 validation before being used for model training

Agentic Coding Interface

```
function: capture_feedback(executed_proposals, parameter_version)

returns: TrainingDataPoint | SkipReason

preconditions:
- proposals.finalized (Layer 2 execution confirmed)
- parameter_version.known and matches active set
```

Training Data Point Structure

```
TrainingDataPoint {
  parameter_version: "model_v3_abc123",
  proposal_count: 12,
  prediction_accuracy: 0.75,
  avg_voter_turnout: 2450,
  avg_time_to_consensus: 8400,
  timestamp: 1714608000,
  outlier_flagged: false
}
```

GATE 8: MODEL VERSIONING & GOVERNANCE GATE

Interface: Meta-Governance Layer

Purpose: To govern the AI model itself. The optimization engine is not a static piece of code — it evolves through retraining on feedback data. However, model updates represent a significant systemic risk: a compromised or poorly trained model could destabilize the entire governance system. This gate ensures that every model version is hash-committed, reproducible, and subject to the same democratic scrutiny as any other proposal. It implements meta-governance: governance over the governance mechanism itself.

Requirements

- All model versions must be hash-committed and reproducible — no black-box models
- Model updates require a separate governance proposal (meta-governance vote)
- Rollback capability to any previous model version in case of degradation
- A/B testing framework for comparing model performance before full activation
- Model artifacts stored on decentralized storage (IPFS/Arweave) with content-addressed hashes

Validation Rules

- ✓ Model SHA-256 hash must match the hash committed in the governance proposal
- ✓ Only governance-approved model versions can be activated by the orchestrator
- ✓ Rollback to a previous version requires Layer 5 veto authority or a new governance vote
- ✓ Active model version must be registered before GATE_3 will accept it
- ✓ Training metadata must include data provenance, hyperparameters, and performance benchmarks

Agentic Coding Interface

```
function: register_model_version(model_artifact, training_metadata,
governance_vote)
```

```
returns: ModelVersionID | RegistrationError
```

```
preconditions:
```

- governance_vote.passed (meta-governance approval)
- model_artifact.hash_verified against committed hash

Model Version Lifecycle

1. Train new model on feedback data (GATE_7 output)
2. Compute SHA-256 hash of model artifact
3. Submit meta-governance proposal with committed hash
4. Community votes on model approval
5. If passed → GATE_8 registers version (inactive by default)
6. Separate activation vote (or veto authority) enables the model
7. GATE_3 now accepts the new version for optimization
8. If performance degrades → GATE_8 rollback to previous version

CORE DATA STRUCTURES

The following data structures form the type system of the Layer 3 Integration Gates. All structures are designed to be immutable where possible, serializable for audit trails, and cryptographically hashable.

ParameterBounds

Immutable definition of safety boundaries for each tunable parameter.

```
@dataclass(frozen=True)
class ParameterBounds:
    param_name: str
    lower_bound: Decimal
    upper_bound: Decimal
    default_value: Decimal
    max_delta_per_epoch: Decimal
    min_safety_floor: Optional[Decimal] = None
    max_safety_ceiling: Optional[Decimal] = None
```

ParameterAdjustment

A single parameter adjustment recommendation from the optimization engine.

```
@dataclass
class ParameterAdjustment:
    param_name: str
    proposed_value: Decimal
    current_value: Decimal
    confidence_score: float # [0.0, 1.0]
    justification: str
    expected_impact: str
```

ParameterSet

The complete state of all governance parameters at any point in time.

```
@dataclass
class ParameterSet:
    quorum_threshold: Decimal = Decimal("0.51")
    voting_duration_blocks: int = 10000
    voice_credit_regeneration_rate: int = 10
    proposal_deposit_amount: int = 1000
    dynamic_quorum_multiplier: Decimal = Decimal("1.0")
```

KPIMetrics

Computed governance health indicators. All values bounded to [0, 1] for consistency.

```
@dataclass
class KPIMetrics:
    voter_apathy_rate: float
    proposal_success_rate: float
```

```
quadratic_efficiency: float
treasury_impact_velocity: float
credit_utilization_rate: float
timestamp: int

def is_valid(self) -> bool:
    return all(0 <= v <= 1 for v in [
        voter_apathy_rate, proposal_success_rate,
        quadratic_efficiency, credit_utilization_rate
    ])
```

OracleSignature

Cryptographic attestation from a decentralized oracle source.

```
@dataclass
class OracleSignature:
    oracle_id: str
    signature: bytes
    timestamp: int
    data_hash: str
```

ModelVersion

Registered and governance-approved AI model artifact.

```
@dataclass
class ModelVersion:
    version_id: str
    model_hash: str
    training_metadata: Dict
    governance_vote_hash: str
    is_active: bool = False
```

LAYER 3 ORCHESTRATOR

The **Layer3Orchestrator** is the master controller that wires all 8 gates together into the autonomous OODA loop. It enforces sequential execution: a failure at any gate halts the cycle and returns a detailed error report. No gate can be bypassed, reordered, or parallelized in a way that skips validation.

OODA Loop Execution Flow

```
ACT → Layer 2: Users submit proposals and cast quadratic votes using Voice Credits
↓
OBSERVE → GATE 1: Layer 4 data is ingested, verified, and validated
↓
ORIENT → GATE 2: KPIs are computed from validated data
→ GATE 3: ML model generates parameter adjustment recommendations
↓
ADAPT → GATE 4: Safety bounds are enforced on all recommendations
→ GATE 5: Treasury isolation is cryptographically verified
→ GATE 6: Safe parameters are broadcast to Layer 5 timelock
↓
VERIFY → Layer 5: Timelock delay expires; veto mechanism cancels or approves
↓
LEARN → GATE 7: Post-execution feedback is captured and validated
→ GATE 8: Model governance tracks versions and enables retraining
```

Cycle Result Structure

Each evolution cycle produces an immutable result record containing the outcome of every gate, enabling full post-hoc auditability of the self-evolution process.

```
{
  "cycle_id": "a3f7e2d9b1c4...",
  "timestamp": 1714608000,
  "status": "submitted_to_timelock",
  "gates": {
    "GATE_1": {"type": "ValidatedDataFrame", "oracle_count": 4},
    "GATE_2": {"type": "KPIMetrics", "voter_apathy_rate": 0.72},
    "GATE_3": {"type": "ParameterAdjustmentProposal", "confidence": 0.85},
    "GATE_4": {"type": "SafeParameterSet", "adjustments": 2},
    "GATE_5": {"type": "IsolatedTx", "isolation_confirmed": true},
    "GATE_6": {"type": "TimelockTxHash", "tx_hash": "0x..."}
  }
}
```

IMPLEMENTATION NOTES FOR AGENTIC CODING

1. Determinism is Mandatory

Every gate must produce deterministic output for identical input. This is not merely a best practice — it is a safety requirement. Non-deterministic KPI computation or optimization results would make audit trails meaningless and introduce attack vectors via timing or randomness manipulation. Use fixed-point arithmetic (Decimal, not float) for all financial calculations.

2. Immutable Audit Trails

Every gate rejection, every safety violation, every parameter adjustment must be logged to an append-only, cryptographically signed audit trail. This trail is the evidence base for post-mortem analysis if the system ever optimizes toward an undesirable state. Store hashes on-chain; store full logs on decentralized storage.

3. Defense in Depth

The 8 gates are designed as defense-in-depth layers. GATE_4 (Safety Bounds) and GATE_5 (Treasury Isolation) are redundant protections — even if the optimization engine is compromised, these gates prevent harmful execution. Similarly, GATE_6 (Timelock) provides a human oversight window even if all automated gates pass. No single point of failure should be able to destabilize the system.

4. Model Governance is Not Optional

GATE_8 is often overlooked in agentic systems, but it is essential. The optimization model itself is a critical system component and must be subject to the same governance rigor as parameter adjustments. Without meta-governance, a compromised model could systematically degrade the DAO over time in ways that individual parameter checks cannot detect.

5. Conservative Fallbacks

When in doubt, the system must fall back to conservative (current) parameters. GATE_3's confidence threshold (0.7) ensures that uncertain optimizations are not applied. This creates a bias toward stability — the system only evolves when it has high confidence that evolution is beneficial. This is the implementation of the Safety Principle.

6. Oracle Diversity

GATE_1 requires ≥ 3 independent oracle sources. Do not use multiple nodes from the same provider (e.g., 3 Chainlink nodes) — this is not true diversity. Use providers with different architectures, data sources, and consensus mechanisms (e.g., Chainlink + API3 + Pyth). This prevents correlated failures from a single oracle compromise.

SAFETY CONSIDERATIONS & ATTACK MITIGATION

Attack Vector: Data Manipulation

Mitigation: GATE_1 requires multi-source aggregation and cryptographic verification. An attacker would need to compromise ≥ 3 independent oracle networks simultaneously. Additionally, anomaly detection flags statistical outliers for manual review.

Attack Vector: Optimization Engine Compromise

Mitigation: Even if the ML model is compromised, GATE_4 enforces hardcoded bounds, GATE_5 prevents treasury access, and GATE_6 imposes a timelock delay. The worst-case outcome is a bounded, reversible parameter shift — not fund theft or proposal execution.

Attack Vector: Runaway Feedback Loop

Mitigation: GATE_4 limits both per-epoch delta and cumulative delta over N epochs. Even if the model repeatedly proposes extreme values, the safety gate clamps the rate of change. The system cannot oscillate violently or drift to an extreme over time.

Attack Vector: Governance Capture via Quorum Reduction

Mitigation: GATE_4 includes a `minimum_safety_quorum` floor (e.g., 0.10). The optimization engine cannot reduce quorum below this absolute threshold, preventing an attacker from lowering the bar until a malicious proposal passes.

Attack Vector: Model Supply Chain Attack

Mitigation: GATE_8 requires hash verification against governance-committed hashes. A compromised model artifact will fail hash matching and be rejected. All training metadata must be disclosed, enabling community audit of the model's training data and hyperparameters.

Attack Vector: Timelock Bypass

Mitigation: GATE_6 enforces a minimum delay period. The timelock contract itself is immutable (Layer 2 constraint: 'Core contract logic must be non-upgradable'). The veto multisig cannot be bypassed by Layer 3 because Layer 3 has no execution authority — it can only recommend.

CONCLUSION

The Layer 3 Integration Gates represent a rigorous, safety-first approach to autonomous governance optimization. By decomposing the self-evolution loop into 8 distinct, sequentially validated checkpoints, the architecture ensures that:

- **Data Integrity:** Only multi-source, cryptographically verified data enters the optimization pipeline (GATE_1)
- **Reproducibility:** All KPIs are computed deterministically from validated inputs (GATE_2)
- **Intelligent Adaptation:** ML-driven parameter recommendations respond to real governance health metrics (GATE_3)
- **Absolute Bounds:** No parameter can ever escape its hardcoded safety cage (GATE_4)
- **Treasury Immunity:** The intelligence layer is physically incapable of accessing funds (GATE_5)
- **Human Oversight:** All changes sit in timelock with veto capability before taking effect (GATE_6)
- **Continuous Learning:** Real-world outcomes feed back into model improvement (GATE_7)
- **Meta-Governance:** The optimizer itself is governed by the same democratic process it optimizes (GATE_8)

This is not merely an engineering specification — it is a **trust architecture**. In a system where code replaces institutions, the boundaries between what is possible and what is permitted must be enforced not by policy, but by mathematics. These 8 gates are the mathematical walls that keep the DAO self-evolving without becoming self-destructive.

"The Optimization Principle: A static rule applied to a dynamic environment eventually fails. Parameters must be treated as variables in a continuous optimization function, not as constants."

"The Safety Principle: Autonomous evolution introduces systemic risk. Any automated parameter adjustment must be mathematically bounded to prevent runaway feedback loops or malicious capture."

— End of Specification —